01 — Data Cleaning & Preprocessing

PM Accelerator — Weather Trend Forecasting Assessment

This notebook:

- Loads the raw Global Weather Repository CSV
- Parses `last_updated` as datetime and sorts ascending
- Logs and imputes missing values (median for numeric columns)
- Flags outliers via IQR on key numeric columns
- Normalizes numeric features with StandardScaler (saved for reuse)
- Saves the cleaned DataFrame to `data/cleaned_weather.csv`

In [1]:
```
import sys, os
sys.path.insert(0, os.path.join(os.getcwd(), '..'))

import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from src.utils import load_raw_data, save_scaler, DATA_DIR, MODELS_DIR
```

1. Load Raw Data

In [2]:
```
df = load_raw_data()
print(f'Shape: {df.shape}')
print(f'Columns ({len(df.columns)}): {list(df.columns)}')
print(f'Date range: {df["last_updated"].min()} → {df["last_updated"].max()}')
df.head()
```

```
Shape: (130783, 41)
Columns (41): ['country', 'location_name', 'latitude', 'longitude', 'timezone', 'last_updated_epoch', 'last_upda
ted', 'temperature_celsius', 'temperature_fahrenheit', 'condition_text', 'wind_mph', 'wind_kph', 'wind_degree',
'wind_direction', 'pressure_mb', 'pressure_in', 'precip_mm', 'precip_in', 'humidity', 'cloud', 'feels_like_celsi
us', 'feels_like_fahrenheit', 'visibility_km', 'visibility_miles', 'uv_index', 'gust_mph', 'gust_kph', 'air_qual
ity_Carbon_Monoxide', 'air_quality_Ozone', 'air_quality_Nitrogen_dioxide', 'air_quality_Sulphur_dioxide', 'air_q
uality_PM2.5', 'air_quality_PM10', 'air_quality_us-epa-index', 'air_quality_gb-defra-index', 'sunrise', 'sunset'
, 'moonrise', 'moonset', 'moon_phase', 'moon_illumination']
Date range: 2024-05-16 01:45:00 → 2026-03-21 19:30:00
```

```
c:\Users\purva\OneDrive\Desktop\PMA\notebooks\..\src\utils.py:33: UserWarning: The argument 'infer_datetime_form
at' is deprecated and will be removed in a future version. A strict version of it is now the default, see https:
//pandas.pydata.org/pdeps/0004-consistent-to-datetime-parsing.html. You can safely remove this argument.
  df["last_updated"] = pd.to_datetime(df["last_updated"], infer_datetime_format=True)
```

Out[2]:

	country	location_name	latitude	longitude	timezone	last_updated_epoch	last_updated	temperature_c
0	Afghanistan	Kabul	34.52	69.18	Asia/Kabul	1715849100	2024-05-16 13:15:00	
1	Albania	Tirana	41.33	19.82	Europe/Tirane	1715849100	2024-05-16 10:45:00	
2	Algeria	Algiers	36.76	3.05	Africa/Algiers	1715849100	2024-05-16 09:45:00	
3	Andorra	Andorra La Vella	42.50	1.52	Europe/Andorra	1715849100	2024-05-16 10:45:00	
4	Angola	Luanda	-8.84	13.23	Africa/Luanda	1715849100	2024-05-16 09:45:00	

5 rows × 41 columns

2. Set Datetime Index (Sorted Ascending)

In [3]:
```
df.sort_values('last_updated', inplace=True)
df.set_index('last_updated', inplace=True)
print(f'Index dtype: {df.index.dtype}')
print(f'First 5 dates:\n{df.index[:5].tolist()}')
df.head()
```

```
Index dtype: datetime64[ns]
First 5 dates:
[Timestamp('2024-05-16 01:45:00'), Timestamp('2024-05-16 02:45:00'), Timestamp('2024-05-16 02:45:00'), Timestamp
('2024-05-16 02:45:00'), Timestamp('2024-05-16 02:45:00')]
```

Out[3]:

last_updated	country	location_name	latitude	longitude	timezone	last_updated_epoch	temperature_
2024-05-16 01:45:00	United States of America	Washington Park	46.60	-120.49	America/Los_Angeles	1715849100	
2024-05-16 02:45:00	El Salvador	San Salvador	13.71	-89.20	America/El_Salvador	1715849100	
2024-05-16 02:45:00	Costa Rica	San Juan	9.97	-84.08	America/Costa_Rica	1715849100	
2024-05-16 02:45:00	Guatemala	Guatemala City	14.62	-90.53	America/Guatemala	1715849100	
2024-05-16 02:45:00	Nicaragua	Managua	12.15	-86.27	America/Managua	1715849100	

5 rows × 40 columns

3. Missing Value Analysis & Imputation

In [4]:
```
null_counts = df.isnull().sum()
null_pct = (df.isnull().sum() / len(df) * 100).round(2)
missing_report = pd.DataFrame({'null_count': null_counts, 'null_pct': null_pct})
missing_report = missing_report[missing_report['null_count'] > 0].sort_values('null_count', ascending=False)

if len(missing_report) > 0:
    print('Columns with missing values:')
    display(missing_report)
else:
    print('No missing values found.')
```

```
No missing values found.
```

In [5]:
```
# Drop rows where more than 50% of values are null
row_null_pct = df.isnull().mean(axis=1)
rows_to_drop = row_null_pct[row_null_pct > 0.5]
if len(rows_to_drop) > 0:
    print(f'Dropping {len(rows_to_drop)} rows with >50% nulls')
    df = df.loc[row_null_pct <= 0.5]

# Impute numeric columns with median
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
print(f'Imputing {len(numeric_cols)} numeric columns with column median...')
for col in numeric_cols:
    if df[col].isnull().any():
        median_val = df[col].median()
        n_imputed = df[col].isnull().sum()
        df[col].fillna(median_val, inplace=True)
        print(f'  {col}: imputed {n_imputed} values with median={median_val:.2f}')

remaining_nulls = df[numeric_cols].isnull().sum().sum()
print(f'\nRemaining nulls in numeric columns: {remaining_nulls}')
```

```
Imputing 30 numeric columns with column median...

Remaining nulls in numeric columns: 0
```

4. IQR Outlier Flagging

Flag outliers using the Interquartile Range method on key numeric columns. **Outliers are NOT dropped** — they are kept for use in the anomaly detection notebook.

In [6]:
```
outlier_cols = ['temperature_celsius', 'precip_mm', 'wind_mph']
df['is_outlier'] = False

for col in outlier_cols:
    if col not in df.columns:
        print(f'  Warning: Column "{col}" not found, skipping')
        continue
    Q1 = df[col].quantile(0.25)
```

```
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    mask = (df[col] < lower) | (df[col] > upper)
    count = mask.sum()
    df.loc[mask, 'is_outlier'] = True
    print(f'  {col}: Q1={Q1:.2f}, Q3={Q3:.2f}, IQR={IQR:.2f}, '
          f'bounds=[{lower:.2f}, {upper:.2f}], outliers={count}')

total_outlier_rows = df['is_outlier'].sum()
print(f'\nTotal rows flagged as outlier: {total_outlier_rows} '
      f'({total_outlier_rows/len(df)*100:.2f}%)')
print('NOTE: Outliers are NOT dropped — they are kept for anomaly detection.')
```

```
  temperature_celsius: Q1=16.10, Q3=28.00, IQR=11.90, bounds=[-1.75, 45.85], outliers=2697
  precip_mm: Q1=0.00, Q3=0.03, IQR=0.03, bounds=[-0.04, 0.07], outliers=24300
  wind_mph: Q1=3.80, Q3=11.00, IQR=7.20, bounds=[-7.00, 21.80], outliers=2096

Total rows flagged as outlier: 28134 (21.51%)
NOTE: Outliers are NOT dropped — they are kept for anomaly detection.
```

5. Feature Normalization (StandardScaler)

In [7]:
```
exclude_from_scaling = ['is_outlier', 'latitude', 'longitude',
                        'last_updated_epoch', 'moon_illumination']
scale_cols = [c for c in numeric_cols if c not in exclude_from_scaling]

scaler = StandardScaler()
df[scale_cols] = scaler.fit_transform(df[scale_cols])
save_scaler(scaler, 'standard_scaler')
print(f'Scaled {len(scale_cols)} columns.')
```

```
Scaler saved → c:\Users\purva\OneDrive\Desktop\PMA\outputs\models\standard_scaler.pkl
Scaled 26 columns.
```

6. Save Cleaned Data

In [8]:
```
# Save scaled version
output_path = os.path.join(DATA_DIR, 'cleaned_weather.csv')
df.to_csv(output_path)
print(f'Saved cleaned DataFrame → {output_path}')
print(f'Final shape: {df.shape}')

# Also save an UNSCALED version for EDA and modelling
print('\nAlso saving unscaled version for EDA/modelling...')
df_unscaled = load_raw_data()
df_unscaled.sort_values('last_updated', inplace=True)
df_unscaled.set_index('last_updated', inplace=True)

# Reapply imputation
for col in numeric_cols:
    if col in df_unscaled.columns and df_unscaled[col].isnull().any():
        df_unscaled[col].fillna(df_unscaled[col].median(), inplace=True)

# Reapply outlier flag
df_unscaled['is_outlier'] = False
for col in outlier_cols:
    if col not in df_unscaled.columns:
        continue
    Q1 = df_unscaled[col].quantile(0.25)
    Q3 = df_unscaled[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    mask = (df_unscaled[col] < lower) | (df_unscaled[col] > upper)
    df_unscaled.loc[mask, 'is_outlier'] = True

unscaled_path = os.path.join(DATA_DIR, 'cleaned_weather_unscaled.csv')
df_unscaled.to_csv(unscaled_path)
print(f'Saved unscaled DataFrame → {unscaled_path}')

print('\n✅ Data cleaning complete!')
```

```
Saved cleaned DataFrame → c:\Users\purva\OneDrive\Desktop\PMA\data\cleaned_weather.csv
Final shape: (130783, 41)

Also saving unscaled version for EDA/modelling...
```

```
c:\Users\purva\OneDrive\Desktop\PMA\notebooks\..\src\utils.py:33: UserWarning: The argument 'infer_datetime_form
at' is deprecated and will be removed in a future version. A strict version of it is now the default, see https:
//pandas.pydata.org/pdeps/0004-consistent-to-datetime-parsing.html. You can safely remove this argument.

  df["last_updated"] = pd.to_datetime(df["last_updated"], infer_datetime_format=True)
Saved unscaled DataFrame → c:\Users\purva\OneDrive\Desktop\PMA\data\cleaned_weather_unscaled.csv

✅ Data cleaning complete!
```

02 — Exploratory Data Analysis (EDA)

PM Accelerator — Weather Trend Forecasting Assessment

This notebook produces **6 required visualizations** saved to `outputs/figures/`:

1. Temperature over time (daily global mean — line plot)
2. Precipitation distribution (histogram + KDE)
3. Correlation heatmap (all numeric columns)
4. Country-wise average temperature (top 20 — horizontal bar)
5. Box plot by continent (temperature grouped by continent)
6. Wind speed vs temperature scatter (colored by precipitation)

In [1]:
```
import sys, os
sys.path.insert(0, os.path.join(os.getcwd(), '..'))

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
from src.utils import DATA_DIR, FIGURES_DIR, apply_plot_style, save_figure, country_to_continent

apply_plot_style()
%matplotlib inline
```

In [2]:
```
# Load data (unscaled for meaningful plots)
df = pd.read_csv(
    os.path.join(DATA_DIR, 'cleaned_weather_unscaled.csv'),
    parse_dates=['last_updated'],
)
df.set_index('last_updated', inplace=True)
df.sort_index(inplace=True)
print(f'Shape: {df.shape}')
df.head()
```

```
Shape: (130783, 41)
```

Out[2]:

last_updated	country	location_name	latitude	longitude	timezone	last_updated_epoch	temperature_
2024-05-16 01:45:00	United States of America	Washington Park	46.60	-120.49	America/Los_Angeles	1715849100	
2024-05-16 02:45:00	El Salvador	San Salvador	13.71	-89.20	America/El_Salvador	1715849100	
2024-05-16 02:45:00	Costa Rica	San Juan	9.97	-84.08	America/Costa_Rica	1715849100	
2024-05-16 02:45:00	Guatemala	Guatemala City	14.62	-90.53	America/Guatemala	1715849100	
2024-05-16 02:45:00	Nicaragua	Managua	12.15	-86.27	America/Managua	1715849100	

5 rows × 41 columns

Plot 1 — Temperature Over Time

Line plot of daily global mean temperature.

In [3]:
```
daily_temp = df['temperature_celsius'].resample('D').mean()

fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(daily_temp.index, daily_temp.values, color='#E74C3C', linewidth=1.0, alpha=0.85)
ax.fill_between(daily_temp.index, daily_temp.values, alpha=0.15, color='#E74C3C')
ax.set_title('Daily Global Mean Temperature Over Time', fontsize=16, fontweight='bold')
ax.set_xlabel('Date')
ax.set_ylabel('Temperature (°C)')
ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=2))
```

```
plt.xticks(rotation=45)
ax.grid(True, alpha=0.3)
save_figure(fig, '01_temperature_over_time.png')
plt.show()
```

```
Figure saved → c:\Users\purva\OneDrive\Desktop\PMA\outputs\figures\01_temperature_over_time.png
```

Plot 2 — Precipitation Distribution

Histogram with KDE overlay (capped at 99th percentile for readability).

In [4]:
```
fig, ax = plt.subplots(figsize=(12, 5))
precip_data = df['precip_mm'].dropna()
cap = precip_data.quantile(0.99)
precip_capped = precip_data[precip_data <= cap]

ax.hist(precip_capped, bins=80, color='#3498DB', edgecolor='white', alpha=0.7, density=True, label='Histogram')
precip_capped.plot.kde(ax=ax, color='#E74C3C', linewidth=2, label='KDE')
ax.set_title('Precipitation Distribution (capped at 99th percentile)', fontsize=16, fontweight='bold')
ax.set_xlabel('Precipitation (mm)')
ax.set_ylabel('Density')
ax.legend()
ax.grid(True, alpha=0.3)
save_figure(fig, '02_precipitation_distribution.png')
plt.show()
```

```
Figure saved → c:\Users\purva\OneDrive\Desktop\PMA\outputs\figures\02_precipitation_distribution.png
```

Plot 3 — Correlation Heatmap

Annotated heatmap of all numeric feature correlations.

In [5]:
```
numeric_df = df.select_dtypes(include=[np.number])
keep = numeric_df.columns[numeric_df.isnull().mean() < 0.3]
corr = numeric_df[keep].corr()

fig, ax = plt.subplots(figsize=(16, 13))
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt='.1f', cmap='RdBu_r',
            center=0, square=True, linewidths=0.5,
            annot_kws={'size': 7}, ax=ax,
            cbar_kws={'shrink': 0.8})
ax.set_title('Correlation Heatmap — Numeric Features', fontsize=16, fontweight='bold', pad=20)
plt.xticks(rotation=45, ha='right', fontsize=8)
plt.yticks(fontsize=8)
save_figure(fig, '03_correlation_heatmap.png')
plt.show()
```

```
Figure saved → c:\Users\purva\OneDrive\Desktop\PMA\outputs\figures\03_correlation_heatmap.png
```

Plot 4 — Country-Wise Average Temperature (Top 20)

Horizontal bar chart of the hottest 20 countries by mean temperature.

In [6]:
```
country_temp = (
    df.groupby('country')['temperature_celsius']
    .mean()
    .sort_values(ascending=True)
    .tail(20)
)

fig, ax = plt.subplots(figsize=(10, 8))
colors = plt.cm.YlOrRd(np.linspace(0.3, 0.95, len(country_temp)))
ax.barh(country_temp.index, country_temp.values, color=colors, edgecolor='white')
ax.set_title('Top 20 Countries by Average Temperature', fontsize=16, fontweight='bold')
ax.set_xlabel('Average Temperature (°C)')
ax.set_ylabel('Country')
for i, (val, name) in enumerate(zip(country_temp.values, country_temp.index)):
    ax.text(val + 0.3, i, f'{val:.1f}°C', va='center', fontsize=9)
ax.grid(True, axis='x', alpha=0.3)
save_figure(fig, '04_country_avg_temperature.png')
plt.show()
```

```
Figure saved → c:\Users\purva\OneDrive\Desktop\PMA\outputs\figures\04_country_avg_temperature.png
```

Plot 5 — Temperature Box Plot by Continent

Temperature distribution grouped by continent, derived from country names.

In [7]:
```
print('Mapping countries to continents...')
unique_countries = df['country'].unique()
continent_cache = {c: country_to_continent(c) for c in unique_countries}
df['continent'] = df['country'].map(continent_cache)

df_cont = df[df['continent'] != 'Unknown'].copy()

fig, ax = plt.subplots(figsize=(12, 6))
continent_order = (
    df_cont.groupby('continent')['temperature_celsius']
    .median()
    .sort_values(ascending=False)
    .index.tolist()
)
palette = sns.color_palette('Set2', len(continent_order))
sns.boxplot(
    data=df_cont, x='continent', y='temperature_celsius',
    order=continent_order, palette=palette, ax=ax,
    fliersize=1, linewidth=1.2,
)
ax.set_title('Temperature Distribution by Continent', fontsize=16, fontweight='bold')
ax.set_xlabel('Continent')
ax.set_ylabel('Temperature (°C)')
ax.grid(True, axis='y', alpha=0.3)
save_figure(fig, '05_temperature_by_continent.png')
plt.show()
```

```
Mapping countries to continents...
Figure saved → c:\Users\purva\OneDrive\Desktop\PMA\outputs\figures\05_temperature_by_continent.png
```

Plot 6 — Wind Speed vs Temperature Scatter

Scatter plot colored by precipitation (mm) using a colormap.

In [8]:
```
sample = df.sample(n=min(15000, len(df)), random_state=42)

fig, ax = plt.subplots(figsize=(12, 6))
scatter = ax.scatter(
    sample['wind_mph'],
    sample['temperature_celsius'],
    c=sample['precip_mm'],
    cmap='YlGnBu',
    alpha=0.5, s=8, edgecolors='none',
    vmin=0, vmax=sample['precip_mm'].quantile(0.95),
)
cbar = plt.colorbar(scatter, ax=ax, shrink=0.8)
cbar.set_label('Precipitation (mm)')
ax.set_title('Wind Speed vs Temperature (colored by Precipitation)', fontsize=16, fontweight='bold')
ax.set_xlabel('Wind Speed (mph)')
ax.set_ylabel('Temperature (°C)')
ax.grid(True, alpha=0.3)
save_figure(fig, '06_wind_vs_temperature_scatter.png')
plt.show()

print('\n✅ EDA complete! 6 plots saved to outputs/figures/')
```

```
Figure saved → c:\Users\purva\OneDrive\Desktop\PMA\outputs\figures\06_wind_vs_temperature_scatter.png

✅ EDA complete! 6 plots saved to outputs/figures/
```

03 — Anomaly Detection

PM Accelerator — Weather Trend Forecasting Assessment

This notebook:

- Selects key weather features for anomaly detection
- Scales features with StandardScaler
- Fits **IsolationForest** (contamination=0.05)
- Labels anomalies in the DataFrame
- Scatter plot of temperature over time colored by anomaly status
- Prints top 10 anomalous rows for inspection

In [1]:
```
import sys, os
sys.path.insert(0, os.path.join(os.getcwd(), '..'))

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
from src.utils import DATA_DIR, FIGURES_DIR, apply_plot_style, save_figure

apply_plot_style()
%matplotlib inline
```

In [2]:
```
df = pd.read_csv(
    os.path.join(DATA_DIR, 'cleaned_weather_unscaled.csv'),
    parse_dates=['last_updated'],
)
# Reset index to avoid duplicate datetime issues
df.sort_values('last_updated', inplace=True)
df.reset_index(drop=True, inplace=True)
print(f'Shape: {df.shape}')
```

```
Shape: (130783, 42)
```

Feature Selection

Select features for anomaly detection, dropping any with >20% missing data.

In [3]:
```
anomaly_features = ['temperature_celsius', 'precip_mm', 'wind_mph', 'humidity', 'pressure_mb']

available = []
for col in anomaly_features:
    if col in df.columns:
        pct_missing = df[col].isnull().mean()
        if pct_missing <= 0.20:
            available.append(col)
            print(f'  ✔ {col} ({pct_missing*100:.1f}% missing)')
        else:
            print(f'  ✗ {col} dropped ({pct_missing*100:.1f}% missing)')

print(f'\nUsing {len(available)} features: {available}')
```

```
  ✔ temperature_celsius (0.0% missing)
  ✔ precip_mm (0.0% missing)
  ✔ wind_mph (0.0% missing)
  ✔ humidity (0.0% missing)
  ✔ pressure_mb (0.0% missing)

Using 5 features: ['temperature_celsius', 'precip_mm', 'wind_mph', 'humidity', 'pressure_mb']
```

Fit Isolation Forest

In [4]:
```
X = df[available].dropna()
print(f'Rows used for anomaly detection: {len(X)}')

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

iso_forest = IsolationForest(
    contamination=0.05,
    random_state=42,
    n_estimators=100,
```

```
    n_jobs=-1,
)
preds = iso_forest.fit_predict(X_scaled)

# Label anomalies — use integer index to avoid duplicate datetime issues
df['anomaly'] = 1  # default: normal
df.loc[X.index, 'anomaly'] = preds

anomaly_count = (df['anomaly'] == -1).sum()
normal_count = (df['anomaly'] == 1).sum()
total = len(df)

print(f'\n{"="*50}')
print(f'ANOMALY DETECTION RESULTS')
print(f'{"="*50}')
print(f'Normal:    {normal_count:>8,} ({normal_count/total*100:.2f}%)')
print(f'Anomaly:   {anomaly_count:>8,} ({anomaly_count/total*100:.2f}%)')
print(f'Total:     {total:>8,}')
```

```
Rows used for anomaly detection: 130783

==================================================
ANOMALY DETECTION RESULTS
==================================================
Normal:     124,243 (95.00%)
Anomaly:      6,540 (5.00%)
Total:      130,783
```

Anomaly Scatter Plot

Temperature over time, with anomalies highlighted in red.

In [5]:
```
fig, ax = plt.subplots(figsize=(14, 6))

normals = df[df['anomaly'] == 1]
anomalies = df[df['anomaly'] == -1]

if len(normals) > 20000:
    normals_plot = normals.sample(20000, random_state=42)
else:
    normals_plot = normals

ax.scatter(normals_plot['last_updated'], normals_plot['temperature_celsius'],
           c='#3498DB', s=2, alpha=0.3, label='Normal', edgecolors='none')
ax.scatter(anomalies['last_updated'], anomalies['temperature_celsius'],
           c='#E74C3C', s=8, alpha=0.7, label='Anomaly', edgecolors='none')

ax.set_title('Temperature Over Time — Anomalies Detected by Isolation Forest',
             fontsize=16, fontweight='bold')
ax.set_xlabel('Date')
ax.set_ylabel('Temperature (°C)')
ax.legend(loc='upper right', fontsize=12, markerscale=3)
ax.grid(True, alpha=0.3)
save_figure(fig, '07_anomaly_detection.png')
plt.show()
```

```
Figure saved → c:\Users\purva\OneDrive\Desktop\PMA\outputs\figures\07_anomaly_detection.png
```

Top 10 Anomalous Rows

In [6]:
```
anomaly_df = df[df['anomaly'] == -1][available + ['country', 'location_name']].copy()
anomaly_df['temp_abs'] = anomaly_df['temperature_celsius'].abs()
top10 = anomaly_df.nlargest(10, 'temp_abs').drop(columns=['temp_abs'])
display(top10)
```

	temperature_celsius	precip_mm	wind_mph	humidity	pressure_mb	country	location_name
6918	49.2	0.0	13.6	4	996.0	Kuwait	Kuwait City
7512	49.1	0.0	6.9	8	994.0	Iraq	Baghdad
11180	49.1	0.0	12.8	6	999.0	Iraq	Baghdad
8455	48.9	0.0	13.6	4	993.0	Kuwait	Kuwait City
11370	48.8	0.0	22.4	6	995.0	Iraq	Baghdad
85071	48.8	0.0	2.2	5	995.0	Kuwait	Kuwait City
76883	48.6	0.0	8.1	4	996.0	Kuwait	Kuwait City
7312	48.4	0.0	10.5	8	998.0	Iraq	Baghdad
10004	48.3	0.0	21.7	6	995.0	Iraq	Baghdad
88202	48.2	0.0	21.5	7	1000.0	Kuwait	Kuwait City

In [7]:
```
# Save updated dataframe
output_path = os.path.join(DATA_DIR, 'weather_with_anomalies.csv')
df.to_csv(output_path, index=False)
print(f'Saved data with anomaly labels → {output_path}')
print('\n✅ Anomaly detection complete!')
```

```
Saved data with anomaly labels → c:\Users\purva\OneDrive\Desktop\PMA\data\weather_with_anomalies.csv

✅ Anomaly detection complete!
```

04 — Forecasting Models

PM Accelerator — Weather Trend Forecasting Assessment

Builds and evaluates three forecasting models + ensemble:

- **Model 1:** ARIMA (2,1,2)
- **Model 2:** Facebook Prophet
- **Model 3:** XGBoost with lag features
- **Ensemble:** Simple average of all three

Target: daily mean `temperature_celsius` (global average)
Split: 80% train / 20% test — chronological
Metrics: MAE, RMSE, MAPE

In [5]:
```
import sys, os
sys.path.insert(0, os.path.join(os.getcwd(), '..'))

import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import joblib
from src.utils import (
    DATA_DIR, FIGURES_DIR, MODELS_DIR, OUTPUT_DIR,
    compute_mae, compute_rmse, compute_mape,
    apply_plot_style, save_figure,
)

apply_plot_style()
%matplotlib inline
```

Load & Prepare Daily Series

In [6]:
```
df = pd.read_csv(
    os.path.join(DATA_DIR, 'cleaned_weather_unscaled.csv'),
    parse_dates=['last_updated'],
)
df.set_index('last_updated', inplace=True)
df.sort_index(inplace=True)

daily = df['temperature_celsius'].resample('D').mean().dropna()
print(f'Daily series: {len(daily)} days')
print(f'Date range: {daily.index.min()} → {daily.index.max()}')
```

```
Daily series: 674 days
Date range: 2024-05-16 00:00:00 → 2026-03-21 00:00:00
```

Train / Test Split (80/20 Chronological)

In [7]:
```
split_idx = int(len(daily) * 0.8)
train = daily.iloc[:split_idx]
test = daily.iloc[split_idx:]
print(f'Train: {len(train)} days ({train.index.min()} → {train.index.max()})')
print(f'Test:  {len(test)} days ({test.index.min()} → {test.index.max()})')

predictions = {}
```

```
Train: 539 days (2024-05-16 00:00:00 → 2025-11-06 00:00:00)
Test:  135 days (2025-11-07 00:00:00 → 2026-03-21 00:00:00)
```

In [8]:
```
pip install statsmodels
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: statsmodels in c:\users\purva\appdata\roaming\python\python312\site-packages (0.1
4.6)
Requirement already satisfied: numpy<3,>=1.22.3 in c:\users\purva\appdata\roaming\python\python312\site-packages
(from statsmodels) (1.26.3)
Requirement already satisfied: scipy!=1.9.2,>=1.8 in c:\users\purva\appdata\roaming\python\python312\site-packag
es (from statsmodels) (1.12.0)
Requirement already satisfied: pandas!=2.1.0,>=1.4 in c:\users\purva\appdata\roaming\python\python312\site-packa
ges (from statsmodels) (2.1.4)
Requirement already satisfied: patsy>=0.5.6 in c:\users\purva\appdata\roaming\python\python312\site-packages (fr
om statsmodels) (1.0.2)
Requirement already satisfied: packaging>=21.3 in c:\users\purva\appdata\roaming\python\python312\site-packages
(from statsmodels) (23.2)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\purva\appdata\roaming\python\python312\site-pa
ckages (from pandas!=2.1.0,>=1.4->statsmodels) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\purva\appdata\roaming\python\python312\site-packages (fr
om pandas!=2.1.0,>=1.4->statsmodels) (2025.2)
Requirement already satisfied: tzdata>=2022.1 in c:\users\purva\appdata\roaming\python\python312\site-packages (
from pandas!=2.1.0,>=1.4->statsmodels) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\purva\appdata\roaming\python\python312\site-packages (from p
ython-dateutil>=2.8.2->pandas!=2.1.0,>=1.4->statsmodels) (1.16.0)
Note: you may need to restart the kernel to use updated packages.
[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Model 1 — ARIMA(2,1,2)

In [9]:
```
from statsmodels.tsa.arima.model import ARIMA

try:
    model_arima = ARIMA(train, order=(2, 1, 2))
    result_arima = model_arima.fit()
    pred_arima = result_arima.forecast(steps=len(test))
    pred_arima.index = test.index
    predictions['ARIMA'] = pred_arima.values

    print(f'MAE:  {compute_mae(test, pred_arima):.4f}')
    print(f'RMSE: {compute_rmse(test, pred_arima):.4f}')
    print(f'MAPE: {compute_mape(test, pred_arima):.4f}%')
    joblib.dump(result_arima, os.path.join(MODELS_DIR, 'arima_model.pkl'))
except Exception as e:
    print(f'ARIMA failed: {e}')
    predictions['ARIMA'] = np.full(len(test), train.mean())
```

```
MAE:  3.5493
RMSE: 3.9109
MAPE: 24.8147%
```

In [10]:
```
pip install prophet 2>&1
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: prophet in c:\users\purva\appdata\roaming\python\python312\site-packages (1.3.0)
Requirement already satisfied: cmdstanpy>=1.0.4 in c:\users\purva\appdata\roaming\python\python312\site-packages
(from prophet) (1.3.0)
Requirement already satisfied: numpy>=1.15.4 in c:\users\purva\appdata\roaming\python\python312\site-packages (f
rom prophet) (1.26.3)
Requirement already satisfied: matplotlib>=2.0.0 in c:\users\purva\appdata\roaming\python\python312\site-package
s (from prophet) (3.8.2)
Requirement already satisfied: pandas>=1.0.4 in c:\users\purva\appdata\roaming\python\python312\site-packages (f
rom prophet) (2.1.4)
Requirement already satisfied: holidays<1,>=0.25 in c:\users\purva\appdata\roaming\python\python312\site-package
s (from prophet) (0.93)
Requirement already satisfied: tqdm>=4.36.1 in c:\users\purva\appdata\roaming\python\python312\site-packages (fr
om prophet) (4.67.1)
Requirement already satisfied: importlib_resources in c:\users\purva\appdata\roaming\python\python312\site-packa
ges (from prophet) (6.5.2)
Requirement already satisfied: python-dateutil<3,>=2.9.0.post0 in c:\users\purva\appdata\roaming\python\python31
2\site-packages (from holidays<1,>=0.25->prophet) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\purva\appdata\roaming\python\python312\site-packages (from p
ython-dateutil<3,>=2.9.0.post0->holidays<1,>=0.25->prophet) (1.16.0)
Requirement already satisfied: stanio<2.0.0,>=0.4.0 in c:\users\purva\appdata\roaming\python\python312\site-pack
ages (from cmdstanpy>=1.0.4->prophet) (0.5.1)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\purva\appdata\roaming\python\python312\site-packages
(from matplotlib>=2.0.0->prophet) (1.2.0)
Requirement already satisfied: cycler>=0.10 in c:\users\purva\appdata\roaming\python\python312\site-packages (fr
om matplotlib>=2.0.0->prophet) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\purva\appdata\roaming\python\python312\site-package
s (from matplotlib>=2.0.0->prophet) (4.47.2)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\purva\appdata\roaming\python\python312\site-package
s (from matplotlib>=2.0.0->prophet) (1.4.5)
Requirement already satisfied: packaging>=20.0 in c:\users\purva\appdata\roaming\python\python312\site-packages
(from matplotlib>=2.0.0->prophet) (23.2)
Requirement already satisfied: pillow>=8 in c:\users\purva\appdata\roaming\python\python312\site-packages (from
matplotlib>=2.0.0->prophet) (10.2.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\purva\appdata\roaming\python\python312\site-packages
(from matplotlib>=2.0.0->prophet) (3.1.1)
Requirement already satisfied: pytz>=2020.1 in c:\users\purva\appdata\roaming\python\python312\site-packages (fr
om pandas>=1.0.4->prophet) (2025.2)
Requirement already satisfied: tzdata>=2022.1 in c:\users\purva\appdata\roaming\python\python312\site-packages (
from pandas>=1.0.4->prophet) (2025.2)
Requirement already satisfied: colorama in c:\users\purva\appdata\roaming\python\python312\site-packages (from t
qdm>=4.36.1->prophet) (0.4.6)

[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.
```

Model 2 — Facebook Prophet

In [11]:

```python
try:
    from prophet import Prophet

    train_prophet = pd.DataFrame({'ds': train.index, 'y': train.values})
    m = Prophet(yearly_seasonality=True, weekly_seasonality=True)
    m.fit(train_prophet)

    future = m.make_future_dataframe(periods=len(test))
    forecast = m.predict(future)

    pred_prophet_full = forecast.set_index('ds')['yhat']
    pred_prophet = pred_prophet_full.reindex(test.index).values
    if np.any(np.isnan(pred_prophet)):
        pred_prophet = forecast['yhat'].values[-len(test):]

    predictions['Prophet'] = pred_prophet
    print(f'MAE:  {compute_mae(test.values, pred_prophet):.4f}')
    print(f'RMSE: {compute_rmse(test.values, pred_prophet):.4f}')
    print(f'MAPE: {compute_mape(test.values, pred_prophet):.4f}%')
    joblib.dump(m, os.path.join(MODELS_DIR, 'prophet_model.pkl'))

except Exception as e:
    print(f'Prophet failed: {e}')
    print('Falling back to SARIMA...')
    try:
        from statsmodels.tsa.statespace.sarimax import SARIMAX
        model_sarima = SARIMAX(train, order=(2, 1, 2),
                               seasonal_order=(1, 1, 1, 7),
                               enforce_stationarity=False,
                               enforce_invertibility=False)
        result_sarima = model_sarima.fit(disp=False)
        pred_sarima = result_sarima.forecast(steps=len(test))
        predictions['Prophet (SARIMA fallback)'] = pred_sarima.values
```

```
        print(f'MAE:  {compute_mae(test, pred_sarima):.4f}')
        print(f'RMSE: {compute_rmse(test, pred_sarima):.4f}')
        print(f'MAPE: {compute_mape(test, pred_sarima):.4f}%')
    except Exception as e2:
        print(f'SARIMA also failed: {e2}')
        predictions['Prophet'] = np.full(len(test), train.mean())
```

```
02:54:02 - cmdstanpy - INFO - Chain [1] start processing
02:54:03 - cmdstanpy - INFO - Chain [1] done processing
MAE:  1.0060
RMSE: 1.5798
MAPE: 8.1736%
```

Model 3 — XGBoost with Lag Features

In [12]:

```
import xgboost as xgb

# Create lag features
df_xgb = pd.DataFrame({'temperature_celsius': daily})
for lag in [1, 3, 7, 14, 30]:
    df_xgb[f'temp_lag_{lag}'] = df_xgb['temperature_celsius'].shift(lag)
df_xgb['rolling_mean_7'] = df_xgb['temperature_celsius'].rolling(7).mean()
df_xgb['rolling_std_7'] = df_xgb['temperature_celsius'].rolling(7).std()
df_xgb['day_of_year'] = df_xgb.index.dayofyear
df_xgb['month'] = df_xgb.index.month
df_xgb.dropna(inplace=True)

feature_cols = [c for c in df_xgb.columns if c != 'temperature_celsius']
X_all = df_xgb[feature_cols]
y_all = df_xgb['temperature_celsius']

# Chronological split
split_date = test.index[0]
X_train_xgb = X_all[X_all.index < split_date]
X_test_xgb = X_all[X_all.index >= split_date]
y_train_xgb = y_all[y_all.index < split_date]
y_test_xgb = y_all[y_all.index >= split_date]

print(f'Training samples: {len(X_train_xgb)}')
print(f'Test samples:     {len(X_test_xgb)}')

model_xgb = xgb.XGBRegressor(
    n_estimators=200, learning_rate=0.05, max_depth=6,
    random_state=42, n_jobs=-1,
)
model_xgb.fit(X_train_xgb, y_train_xgb,
              eval_set=[(X_test_xgb, y_test_xgb)], verbose=False)

pred_xgb_full = model_xgb.predict(X_test_xgb)
pred_xgb_series = pd.Series(pred_xgb_full, index=X_test_xgb.index)
pred_xgb_aligned = pred_xgb_series.reindex(test.index).ffill().bfill()
predictions['XGBoost'] = pred_xgb_aligned.values

print(f'MAE:  {compute_mae(test.values, predictions["XGBoost"]):.4f}')
print(f'RMSE: {compute_rmse(test.values, predictions["XGBoost"]):.4f}')
print(f'MAPE: {compute_mape(test.values, predictions["XGBoost"]):.4f}%')
joblib.dump(model_xgb, os.path.join(MODELS_DIR, 'xgboost_model.pkl'))
```

```
Training samples: 509
Test samples:     135
MAE:  1.2332
RMSE: 1.8496
MAPE: 9.9384%
```

Out[12]:

```
['c:\\Users\\purva\\OneDrive\\Desktop\\PMA\\outputs\\models\\xgboost_model.pkl']
```

Ensemble — Simple Average

In [13]:

```
pred_arrays = [v for v in predictions.values()]
min_len = min(len(a) for a in pred_arrays)
pred_arrays_trunc = [a[:min_len] for a in pred_arrays]
test_trunc = test.values[:min_len]

ensemble_pred = np.mean(pred_arrays_trunc, axis=0)
predictions['Ensemble'] = ensemble_pred

print(f'Ensemble MAE:  {compute_mae(test_trunc, ensemble_pred):.4f}')
print(f'Ensemble RMSE: {compute_rmse(test_trunc, ensemble_pred):.4f}')
print(f'Ensemble MAPE: {compute_mape(test_trunc, ensemble_pred):.4f}%')
```

```
Ensemble MAE:  1.8674
Ensemble RMSE: 2.3234
Ensemble MAPE: 13.9587%
```

Model Comparison Table

In [14]:
```python
comparison_dict = {}
for name, preds in predictions.items():
    preds_t = preds[:min_len]
    comparison_dict[name] = {
        'MAE': round(compute_mae(test_trunc, preds_t), 4),
        'RMSE': round(compute_rmse(test_trunc, preds_t), 4),
        'MAPE (%)': round(compute_mape(test_trunc, preds_t), 4),
    }

comparison_df = pd.DataFrame(comparison_dict).T
comparison_df.index.name = 'Model'
display(comparison_df)

comp_path = os.path.join(OUTPUT_DIR, 'model_comparison.csv')
comparison_df.to_csv(comp_path)
print(f'\nComparison saved → {comp_path}')
```

Model	MAE	RMSE	MAPE (%)
ARIMA	3.5493	3.9109	24.8147
Prophet	1.0060	1.5798	8.1736
XGBoost	1.2332	1.8496	9.9384
Ensemble	1.8674	2.3234	13.9587

```
Comparison saved → c:\Users\purva\OneDrive\Desktop\PMA\outputs\model_comparison.csv
```

Forecast Plots

In [15]:
```python
# All models comparison
fig, ax = plt.subplots(figsize=(14, 6))
ax.plot(test.index[:min_len], test_trunc, color='black', linewidth=1.5, label='Actual', alpha=0.8)

colors = {'ARIMA': '#E74C3C', 'Prophet': '#3498DB', 'Prophet (SARIMA fallback)': '#3498DB',
          'XGBoost': '#2ECC71', 'Ensemble': '#9B59B6'}

for name, preds in predictions.items():
    if name == 'Ensemble':
        continue
    ax.plot(test.index[:min_len], preds[:min_len], linewidth=1.0, alpha=0.7,
            label=name, color=colors.get(name, '#95A5A6'))

ax.plot(test.index[:min_len], ensemble_pred, linewidth=2.0, alpha=0.9,
        label='Ensemble', color='#9B59B6', linestyle='--')

ax.set_title('Temperature Forecast — Model Comparison', fontsize=16, fontweight='bold')
ax.set_xlabel('Date')
ax.set_ylabel('Temperature (°C)')
ax.legend(loc='best')
ax.grid(True, alpha=0.3)
save_figure(fig, '08_forecast_comparison.png')
plt.show()
```

```
Figure saved → c:\Users\purva\OneDrive\Desktop\PMA\outputs\figures\08_forecast_comparison.png
```

In [16]:
```python
# Full timeline
fig, ax = plt.subplots(figsize=(14, 6))
ax.plot(train.index, train.values, color='#3498DB', linewidth=0.8, alpha=0.6, label='Train')
ax.plot(test.index[:min_len], test_trunc, color='black', linewidth=1.5, label='Actual (Test)')
ax.plot(test.index[:min_len], ensemble_pred, color='#E74C3C', linewidth=1.5,
        linestyle='--', label='Ensemble Forecast')
ax.axvline(x=test.index[0], color='gray', linestyle=':', alpha=0.7)
ax.set_title('Full Timeline — Train, Test & Ensemble Forecast', fontsize=16, fontweight='bold')
ax.set_xlabel('Date')
ax.set_ylabel('Temperature (°C)')
ax.legend(loc='best')
ax.grid(True, alpha=0.3)
save_figure(fig, '09_full_timeline_forecast.png')
plt.show()

print('\n✅ Forecasting complete!')
```

```
Figure saved → c:\Users\purva\OneDrive\Desktop\PMA\outputs\figures\09_full_timeline_forecast.png

✅ Forecasting complete!
```

05 — Unique Analyses

PM Accelerator — Weather Trend Forecasting Assessment

This notebook implements two unique analyses:

- **A) Feature Importance** — XGBoost feature importances + optional SHAP
- **B) Spatial / Geographical Analysis** — Plotly choropleth world map

In [1]:
```
import sys, os
sys.path.insert(0, os.path.join(os.getcwd(), '..'))

import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import joblib
from src.utils import DATA_DIR, FIGURES_DIR, MODELS_DIR, apply_plot_style, save_figure

apply_plot_style()
%matplotlib inline
```

In [2]:
```
df = pd.read_csv(
    os.path.join(DATA_DIR, 'cleaned_weather_unscaled.csv'),
    parse_dates=['last_updated'],
)
df.set_index('last_updated', inplace=True)
df.sort_index(inplace=True)
print(f'Shape: {df.shape}')
df.head()
```

```
Shape: (130783, 41)
```

Out[2]:

	country	location_name	latitude	longitude	timezone	last_updated_epoch	temperature_
last_updated							
2024-05-16 01:45:00	United States of America	Washington Park	46.60	-120.49	America/Los_Angeles	1715849100	
2024-05-16 02:45:00	El Salvador	San Salvador	13.71	-89.20	America/El_Salvador	1715849100	
2024-05-16 02:45:00	Costa Rica	San Juan	9.97	-84.08	America/Costa_Rica	1715849100	
2024-05-16 02:45:00	Guatemala	Guatemala City	14.62	-90.53	America/Guatemala	1715849100	
2024-05-16 02:45:00	Nicaragua	Managua	12.15	-86.27	America/Managua	1715849100	

5 rows × 41 columns

Analysis A — Feature Importance (XGBoost)

Train an XGBoost model to predict `temperature_celsius` from all other numeric features, then visualize which features contribute most.

Note: We exclude temperature-derived features (`temperature_fahrenheit`, `feels_like_celsius`, `feels_like_fahrenheit`) since they are trivially correlated with the target.

In [3]:
```
import xgboost as xgb

numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
target = 'temperature_celsius'

# Exclude temperature-derived features (leaky), outlier flags, and anomaly labels
exclude_features = [
```

```
    target,
    'temperature_fahrenheit',
    'feels_like_celsius',
    'feels_like_fahrenheit',
    'is_outlier',
]
feature_cols = [c for c in numeric_cols
                if c not in exclude_features and 'anomaly' not in c.lower()]
feature_cols = [c for c in feature_cols if df[c].isnull().mean() < 0.3]

X = df[feature_cols].copy()
y = df[target].copy()
mask = X.isnull().any(axis=1) | y.isnull()
X = X[~mask]
y = y[~mask]

print(f'Features: {len(feature_cols)}')
print(f'Samples:  {len(X)}')
print(f'\nFeatures used: {feature_cols}')

model = xgb.XGBRegressor(
    n_estimators=200, learning_rate=0.05, max_depth=6,
    random_state=42, n_jobs=-1,
)
model.fit(X, y, verbose=False)

importances = pd.Series(model.feature_importances_, index=feature_cols)
importances = importances.sort_values(ascending=True)
top15 = importances.tail(15)
print('\nTop 15 features:')
display(top15.sort_values(ascending=False))
```

```
Features: 26
Samples:  130783

Features used: ['latitude', 'longitude', 'last_updated_epoch', 'wind_mph', 'wind_kph', 'wind_degree', 'pressure_
mb', 'pressure_in', 'precip_mm', 'precip_in', 'humidity', 'cloud', 'visibility_km', 'visibility_miles', 'uv_inde
x', 'gust_mph', 'gust_kph', 'air_quality_Carbon_Monoxide', 'air_quality_Ozone', 'air_quality_Nitrogen_dioxide',
'air_quality_Sulphur_dioxide', 'air_quality_PM2.5', 'air_quality_PM10', 'air_quality_us-epa-index', 'air_quality
_gb-defra-index', 'moon_illumination']

Top 15 features:
uv_index                        0.319935
pressure_in                     0.222813
latitude                        0.190222
last_updated_epoch              0.046776
humidity                        0.036201
longitude                       0.030823
gust_kph                        0.023212
air_quality_Sulphur_dioxide     0.018840
air_quality_Nitrogen_dioxide    0.016652
cloud                           0.012813
air_quality_Carbon_Monoxide     0.012454
wind_mph                        0.010655
air_quality_PM10                0.009980
visibility_km                   0.009250
air_quality_PM2.5               0.007983
dtype: float32
```

In [4]:

```
fig, ax = plt.subplots(figsize=(10, 8))
colors = plt.cm.viridis(np.linspace(0.3, 0.9, len(top15)))
ax.barh(top15.index, top15.values, color=colors, edgecolor='white')
ax.set_title('Top 15 Most Predictive Features for Temperature', fontsize=16, fontweight='bold')
ax.set_xlabel('Feature Importance (Gain)')
ax.grid(True, axis='x', alpha=0.3)
for i, (val, name) in enumerate(zip(top15.values, top15.index)):
    ax.text(val + 0.002, i, f'{val:.3f}', va='center', fontsize=9)
save_figure(fig, 'feature_importance.png')
plt.show()
```

```
Figure saved → c:\Users\purva\OneDrive\Desktop\PMA\outputs\figures\feature_importance.png
```

Optional: SHAP Analysis

In [5]:

```
try:
    import shap
    print('Generating SHAP values (this may take a moment)...')
    explainer = shap.TreeExplainer(model)
    X_sample = X.sample(min(1000, len(X)), random_state=42)
    shap_values = explainer.shap_values(X_sample)

    fig, ax = plt.subplots(figsize=(10, 8))
    shap.summary_plot(shap_values, X_sample, show=False, max_display=15)
```

```
    plt.title('SHAP Feature Importance — Beeswarm Plot', fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.savefig(os.path.join(FIGURES_DIR, 'feature_importance_shap.png'), bbox_inches='tight', dpi=150)
    plt.show()
    print('SHAP plot saved!')
except ImportError:
    print('SHAP not installed — skipping SHAP analysis')
except Exception as e:
    print(f'SHAP analysis failed: {e}')
```

```
SHAP not installed — skipping SHAP analysis
```

In [6]:
```
pip install plotly
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: plotly in c:\users\purva\appdata\roaming\python\python312\site-packages (6.6.0)
Requirement already satisfied: narwhals>=1.15.1 in c:\users\purva\appdata\roaming\python\python312\site-packages
(from plotly) (2.15.0)
Requirement already satisfied: packaging in c:\users\purva\appdata\roaming\python\python312\site-packages (from
plotly) (23.2)
Note: you may need to restart the kernel to use updated packages.
[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [9]:
```
pip install pycountry-convert
```

```
Defaulting to user installation because normal site-packages is not writeable
Collecting pycountry-convert
  Downloading pycountry_convert-0.7.2-py3-none-any.whl.metadata (7.2 kB)
Collecting pprintpp>=0.3.0 (from pycountry-convert)
  Downloading pprintpp-0.4.0-py2.py3-none-any.whl.metadata (7.9 kB)
Collecting pycountry>=16.11.27.1 (from pycountry-convert)
  Downloading pycountry-26.2.16-py3-none-any.whl.metadata (12 kB)
Collecting pytest>=3.4.0 (from pycountry-convert)
  Downloading pytest-9.0.2-py3-none-any.whl.metadata (7.6 kB)
Collecting pytest-mock>=1.6.3 (from pycountry-convert)
  Downloading pytest_mock-3.15.1-py3-none-any.whl.metadata (3.9 kB)
Collecting pytest-cov>=2.5.1 (from pycountry-convert)
  Downloading pytest_cov-7.1.0-py3-none-any.whl.metadata (32 kB)
Collecting repoze.lru>=0.7 (from pycountry-convert)
  Downloading repoze.lru-0.7-py3-none-any.whl.metadata (1.1 kB)
Requirement already satisfied: wheel>=0.30.0 in c:\users\purva\appdata\roaming\python\python312\site-packages (f
rom pycountry-convert) (0.45.1)
Requirement already satisfied: colorama>=0.4 in c:\users\purva\appdata\roaming\python\python312\site-packages (f
rom pytest>=3.4.0->pycountry-convert) (0.4.6)
Collecting iniconfig>=1.0.1 (from pytest>=3.4.0->pycountry-convert)
  Downloading iniconfig-2.3.0-py3-none-any.whl.metadata (2.5 kB)
Requirement already satisfied: packaging>=22 in c:\users\purva\appdata\roaming\python\python312\site-packages (f
rom pytest>=3.4.0->pycountry-convert) (23.2)
Collecting pluggy<2,>=1.5 (from pytest>=3.4.0->pycountry-convert)
  Downloading pluggy-1.6.0-py3-none-any.whl.metadata (4.8 kB)
Requirement already satisfied: pygments>=2.7.2 in c:\users\purva\appdata\roaming\python\python312\site-packages
(from pytest>=3.4.0->pycountry-convert) (2.19.1)
Collecting coverage>=7.10.6 (from coverage[toml]>=7.10.6->pytest-cov>=2.5.1->pycountry-convert)
  Downloading coverage-7.13.5-cp312-cp312-win_amd64.whl.metadata (8.7 kB)
Downloading pycountry_convert-0.7.2-py3-none-any.whl (13 kB)
Downloading pprintpp-0.4.0-py2.py3-none-any.whl (16 kB)
Downloading pycountry-26.2.16-py3-none-any.whl (8.0 MB)
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/8.0 MB ? eta -:--:--
```

```
---------------------------------------- 0.0/8.0 MB ? eta -:--:--
---------------------------------------- 0.0/8.0 MB ? eta -:--:--
---------------------------------------- 0.0/8.0 MB ? eta -:--:--
---------------------------------------- 0.0/8.0 MB ? eta -:--:--
---------------------------------------- 0.0/8.0 MB ? eta -:--:--
---------------------------------------- 0.0/8.0 MB ? eta -:--:--
---------------------------------------- 0.0/8.0 MB ? eta -:--:--
---------------------------------------- 0.0/8.0 MB ? eta -:--:--
---------------------------------------- 0.0/8.0 MB ? eta -:--:--
---------------------------------------- 0.0/8.0 MB ? eta -:--:--
---------------------------------------- 0.0/8.0 MB ? eta -:--:--
---------------------------------------- 0.0/8.0 MB ? eta -:--:--
---------------------------------------- 0.0/8.0 MB ? eta -:--:--
---------------------------------------- 0.0/8.0 MB ? eta -:--:--
---------------------------------------- 0.0/8.0 MB ? eta -:--:--
---------------------------------------- 0.0/8.0 MB ? eta -:--:--
---------------------------------------- 0.0/8.0 MB ? eta -:--:--
---------------------------------------- 0.0/8.0 MB ? eta -:--:--
- -------------------------------------- 0.3/8.0 MB ? eta -:--:--
- -------------------------------------- 0.3/8.0 MB ? eta -:--:--
- -------------------------------------- 0.3/8.0 MB ? eta -:--:--
- -------------------------------------- 0.3/8.0 MB ? eta -:--:--
- -------------------------------------- 0.3/8.0 MB ? eta -:--:--
- -------------------------------------- 0.3/8.0 MB ? eta -:--:--
- -------------------------------------- 0.3/8.0 MB ? eta -:--:--
- -------------------------------------- 0.3/8.0 MB ? eta -:--:--
- -------------------------------------- 0.3/8.0 MB ? eta -:--:--
- -------------------------------------- 0.3/8.0 MB ? eta -:--:--
- -------------------------------------- 0.3/8.0 MB ? eta -:--:--
- -------------------------------------- 0.3/8.0 MB ? eta -:--:--
- -------------------------------------- 0.3/8.0 MB ? eta -:--:--
- -------------------------------------- 0.3/8.0 MB ? eta -:--:--
-- ------------------------------------- 0.5/8.0 MB 89.3 kB/s eta 0:01:25
-- ------------------------------------- 0.5/8.0 MB 89.3 kB/s eta 0:01:25
-- ------------------------------------- 0.5/8.0 MB 89.3 kB/s eta 0:01:25
-- ------------------------------------- 0.5/8.0 MB 89.3 kB/s eta 0:01:25
-- ------------------------------------- 0.5/8.0 MB 89.3 kB/s eta 0:01:25
-- ------------------------------------- 0.5/8.0 MB 89.3 kB/s eta 0:01:25
-- ------------------------------------- 0.5/8.0 MB 89.3 kB/s eta 0:01:25
--- ------------------------------------ 0.8/8.0 MB 116.5 kB/s eta 0:01:03
--- ------------------------------------ 0.8/8.0 MB 116.5 kB/s eta 0:01:03
--- ------------------------------------ 0.8/8.0 MB 116.5 kB/s eta 0:01:03
--- ------------------------------------ 0.8/8.0 MB 116.5 kB/s eta 0:01:03
----- ---------------------------------- 1.0/8.0 MB 146.3 kB/s eta 0:00:48
----- ---------------------------------- 1.0/8.0 MB 146.3 kB/s eta 0:00:48
----- ---------------------------------- 1.0/8.0 MB 146.3 kB/s eta 0:00:48
----- ---------------------------------- 1.0/8.0 MB 146.3 kB/s eta 0:00:48
----- ---------------------------------- 1.0/8.0 MB 146.3 kB/s eta 0:00:48
------ --------------------------------- 1.3/8.0 MB 163.3 kB/s eta 0:00:42
------ --------------------------------- 1.3/8.0 MB 163.3 kB/s eta 0:00:42
------ --------------------------------- 1.3/8.0 MB 163.3 kB/s eta 0:00:42
------ --------------------------------- 1.3/8.0 MB 163.3 kB/s eta 0:00:42
------ --------------------------------- 1.3/8.0 MB 163.3 kB/s eta 0:00:42
------- -------------------------------- 1.6/8.0 MB 175.5 kB/s eta 0:00:37
------- -------------------------------- 1.6/8.0 MB 175.5 kB/s eta 0:00:37
------- -------------------------------- 1.6/8.0 MB 175.5 kB/s eta 0:00:37
------- -------------------------------- 1.6/8.0 MB 175.5 kB/s eta 0:00:37
--------- ------------------------------ 1.8/8.0 MB 188.2 kB/s eta 0:00:34
--------- ------------------------------ 1.8/8.0 MB 188.2 kB/s eta 0:00:34
--------- ------------------------------ 1.8/8.0 MB 188.2 kB/s eta 0:00:34
--------- ------------------------------ 1.8/8.0 MB 188.2 kB/s eta 0:00:34
---------- ----------------------------- 2.1/8.0 MB 199.7 kB/s eta 0:00:30
---------- ----------------------------- 2.1/8.0 MB 199.7 kB/s eta 0:00:30
---------- ----------------------------- 2.1/8.0 MB 199.7 kB/s eta 0:00:30
---------- ----------------------------- 2.1/8.0 MB 199.7 kB/s eta 0:00:30
----------- ---------------------------- 2.4/8.0 MB 209.1 kB/s eta 0:00:28
----------- ---------------------------- 2.4/8.0 MB 209.1 kB/s eta 0:00:28
----------- ---------------------------- 2.4/8.0 MB 209.1 kB/s eta 0:00:28
----------- ---------------------------- 2.4/8.0 MB 209.1 kB/s eta 0:00:28
----------- ---------------------------- 2.4/8.0 MB 209.1 kB/s eta 0:00:28
------------- -------------------------- 2.6/8.0 MB 213.9 kB/s eta 0:00:26
------------- -------------------------- 2.6/8.0 MB 213.9 kB/s eta 0:00:26
------------- -------------------------- 2.6/8.0 MB 213.9 kB/s eta 0:00:26
------------- -------------------------- 2.6/8.0 MB 213.9 kB/s eta 0:00:26
-------------- ------------------------- 2.9/8.0 MB 221.3 kB/s eta 0:00:24
-------------- ------------------------- 2.9/8.0 MB 221.3 kB/s eta 0:00:24
-------------- ------------------------- 2.9/8.0 MB 221.3 kB/s eta 0:00:24
--------------- ------------------------ 3.1/8.0 MB 230.7 kB/s eta 0:00:22
--------------- ------------------------ 3.1/8.0 MB 230.7 kB/s eta 0:00:22
--------------- ------------------------ 3.1/8.0 MB 230.7 kB/s eta 0:00:22
--------------- ------------------------ 3.1/8.0 MB 230.7 kB/s eta 0:00:22
--------------- ------------------------ 3.1/8.0 MB 230.7 kB/s eta 0:00:22
---------------- ----------------------- 3.4/8.0 MB 235.2 kB/s eta 0:00:20
```

```
   ---------------- ----------------------- 3.4/8.0 MB 235.2 kB/s eta 0:00:20
   ---------------- ----------------------- 3.4/8.0 MB 235.2 kB/s eta 0:00:20
   ---------------- ----------------------- 3.4/8.0 MB 235.2 kB/s eta 0:00:20
   ---------------- ----------------------- 3.4/8.0 MB 235.2 kB/s eta 0:00:20
   ------------------ --------------------- 3.7/8.0 MB 235.5 kB/s eta 0:00:19
   ------------------ --------------------- 3.7/8.0 MB 235.5 kB/s eta 0:00:19
   ------------------ --------------------- 3.7/8.0 MB 235.5 kB/s eta 0:00:19
   ------------------ --------------------- 3.7/8.0 MB 235.5 kB/s eta 0:00:19
   ------------------- -------------------- 3.9/8.0 MB 240.2 kB/s eta 0:00:18
   ------------------- -------------------- 3.9/8.0 MB 240.2 kB/s eta 0:00:18
   ------------------- -------------------- 3.9/8.0 MB 240.2 kB/s eta 0:00:18
   ------------------- -------------------- 3.9/8.0 MB 240.2 kB/s eta 0:00:18
   ------------------- -------------------- 3.9/8.0 MB 240.2 kB/s eta 0:00:18
   -------------------- ------------------- 4.2/8.0 MB 241.8 kB/s eta 0:00:16
   -------------------- ------------------- 4.2/8.0 MB 241.8 kB/s eta 0:00:16
   -------------------- ------------------- 4.2/8.0 MB 241.8 kB/s eta 0:00:16
   -------------------- ------------------- 4.2/8.0 MB 241.8 kB/s eta 0:00:16
   ---------------------- ----------------- 4.5/8.0 MB 245.2 kB/s eta 0:00:15
   ---------------------- ----------------- 4.5/8.0 MB 245.2 kB/s eta 0:00:15
   ----------------------- ---------------- 4.7/8.0 MB 252.2 kB/s eta 0:00:14
   ----------------------- ---------------- 4.7/8.0 MB 252.2 kB/s eta 0:00:14
   ----------------------- ---------------- 4.7/8.0 MB 252.2 kB/s eta 0:00:14
   ------------------------ --------------- 5.0/8.0 MB 258.6 kB/s eta 0:00:12
   ------------------------ --------------- 5.0/8.0 MB 258.6 kB/s eta 0:00:12
   ------------------------ --------------- 5.0/8.0 MB 258.6 kB/s eta 0:00:12
   ------------------------ --------------- 5.0/8.0 MB 258.6 kB/s eta 0:00:12
   ------------------------- -------------- 5.2/8.0 MB 262.4 kB/s eta 0:00:11
   ------------------------- -------------- 5.2/8.0 MB 262.4 kB/s eta 0:00:11
   ------------------------- -------------- 5.2/8.0 MB 262.4 kB/s eta 0:00:11
   -------------------------- ------------- 5.5/8.0 MB 266.1 kB/s eta 0:00:10
   -------------------------- ------------- 5.5/8.0 MB 266.1 kB/s eta 0:00:10
   -------------------------- ------------- 5.5/8.0 MB 266.1 kB/s eta 0:00:10
   -------------------------- ------------- 5.5/8.0 MB 266.1 kB/s eta 0:00:10
   -------------------------- ------------- 5.5/8.0 MB 266.1 kB/s eta 0:00:10
   -------------------------- ------------- 5.5/8.0 MB 266.1 kB/s eta 0:00:10
   --------------------------- ------------ 5.8/8.0 MB 263.7 kB/s eta 0:00:09
   --------------------------- ------------ 5.8/8.0 MB 263.7 kB/s eta 0:00:09
   --------------------------- ------------ 5.8/8.0 MB 263.7 kB/s eta 0:00:09
   --------------------------- ------------ 5.8/8.0 MB 263.7 kB/s eta 0:00:09
   --------------------------- ------------ 5.8/8.0 MB 263.7 kB/s eta 0:00:09
   --------------------------- ------------ 5.8/8.0 MB 263.7 kB/s eta 0:00:09
   ---------------------------- ----------- 6.0/8.0 MB 260.3 kB/s eta 0:00:08
   ---------------------------- ----------- 6.0/8.0 MB 260.3 kB/s eta 0:00:08
   ---------------------------- ----------- 6.0/8.0 MB 260.3 kB/s eta 0:00:08
   ---------------------------- ----------- 6.0/8.0 MB 260.3 kB/s eta 0:00:08
   ---------------------------- ----------- 6.0/8.0 MB 260.3 kB/s eta 0:00:08
   ----------------------------- ---------- 6.3/8.0 MB 260.9 kB/s eta 0:00:07
   ----------------------------- ---------- 6.3/8.0 MB 260.9 kB/s eta 0:00:07
   ----------------------------- ---------- 6.3/8.0 MB 260.9 kB/s eta 0:00:07
   ------------------------------ --------- 6.6/8.0 MB 263.7 kB/s eta 0:00:06
   ------------------------------ --------- 6.6/8.0 MB 263.7 kB/s eta 0:00:06
   ------------------------------ --------- 6.6/8.0 MB 263.7 kB/s eta 0:00:06
   ------------------------------ --------- 6.6/8.0 MB 263.7 kB/s eta 0:00:06
   ------------------------------- -------- 6.8/8.0 MB 266.5 kB/s eta 0:00:05
   ------------------------------- -------- 6.8/8.0 MB 266.5 kB/s eta 0:00:05
   -------------------------------- ------- 7.1/8.0 MB 270.9 kB/s eta 0:00:04
   -------------------------------- ------- 7.1/8.0 MB 270.9 kB/s eta 0:00:04
   -------------------------------- ------- 7.1/8.0 MB 270.9 kB/s eta 0:00:04
   --------------------------------- ------ 7.3/8.0 MB 275.0 kB/s eta 0:00:03
   --------------------------------- ------ 7.3/8.0 MB 275.0 kB/s eta 0:00:03
   --------------------------------- ------ 7.3/8.0 MB 275.0 kB/s eta 0:00:03
   ---------------------------------- ----- 7.6/8.0 MB 278.6 kB/s eta 0:00:02
   ---------------------------------- ----- 7.6/8.0 MB 278.6 kB/s eta 0:00:02
   ----------------------------------- ---- 7.9/8.0 MB 283.2 kB/s eta 0:00:01
   ----------------------------------- ---- 7.9/8.0 MB 283.2 kB/s eta 0:00:01
   ---------------------------------------- 8.0/8.0 MB 287.1 kB/s  0:00:36
Downloading pytest-9.0.2-py3-none-any.whl (374 kB)
Downloading pluggy-1.6.0-py3-none-any.whl (20 kB)
Downloading iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
Downloading pytest_cov-7.1.0-py3-none-any.whl (22 kB)
Downloading coverage-7.13.5-cp312-cp312-win_amd64.whl (222 kB)
Downloading pytest_mock-3.15.1-py3-none-any.whl (10 kB)
Downloading repoze.lru-0.7-py3-none-any.whl (10 kB)
Installing collected packages: repoze.lru, pprintpp, pycountry, pluggy, iniconfig, coverage, pytest, pytest-mock
, pytest-cov, pycountry-convert

   -------- -------------------------------  2/10 [pycountry]
   -------- -------------------------------  2/10 [pycountry]
   ------------ ---------------------------  3/10 [pluggy]
   -------------------- -------------------  5/10 [coverage]
   -------------------- -------------------  5/10 [coverage]
   ------------------------ ---------------  6/10 [pytest]
   ------------------------ ---------------  6/10 [pytest]
```

```
   ------------------------ ---------------  6/10 [pytest]
   ------------------------ ---------------  6/10 [pytest]
   ------------------------ ---------------  6/10 [pytest]
   ------------------------ ---------------  6/10 [pytest]
   ------------------------------- -------  8/10 [pytest-cov]
   ---------------------------------------- 10/10 [pycountry-convert]

Successfully installed coverage-7.13.5 iniconfig-2.3.0 pluggy-1.6.0 pprintpp-0.4.0 pycountry-26.2.16 pycountry-c
onvert-0.7.2 pytest-9.0.2 pytest-cov-7.1.0 pytest-mock-3.15.1 repoze.lru-0.7
Note: you may need to restart the kernel to use updated packages.
[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Analysis B — Spatial / Geographical Analysis (Choropleth)

Interactive world map showing mean temperature by country using Plotly.

In [7]:
```
import plotly.express as px

country_stats = (
    df.groupby('country')
    .agg(
        mean_temp=('temperature_celsius', 'mean'),
        mean_humidity=('humidity', 'mean'),
        mean_precip=('precip_mm', 'mean'),
        record_count=('temperature_celsius', 'count'),
    )
    .reset_index()
)
print(f'Countries: {len(country_stats)}')

# Get ISO alpha-3 codes
try:
    import pycountry
    def get_iso3(country_name):
        try:
            result = pycountry.countries.search_fuzzy(country_name)
            return result[0].alpha_3
        except Exception:
            return None
    country_stats['iso_alpha'] = country_stats['country'].apply(get_iso3)
    country_stats = country_stats.dropna(subset=['iso_alpha'])
except ImportError:
    print('pycountry not available, using country names directly')
    country_stats['iso_alpha'] = country_stats['country']
```

```
Countries: 211
```

In [12]:
```
pip install kaleido
```

```
Defaulting to user installation because normal site-packages is not writeable
Collecting kaleido
  Downloading kaleido-1.2.0-py3-none-any.whl.metadata (5.6 kB)
Collecting choreographer>=1.1.1 (from kaleido)
  Downloading choreographer-1.2.1-py3-none-any.whl.metadata (6.8 kB)
Collecting logistro>=1.0.8 (from kaleido)
  Downloading logistro-2.0.1-py3-none-any.whl.metadata (3.9 kB)
Requirement already satisfied: orjson>=3.10.15 in c:\users\purva\appdata\roaming\python\python312\site-packages
(from kaleido) (3.11.7)
Requirement already satisfied: packaging in c:\users\purva\appdata\roaming\python\python312\site-packages (from
kaleido) (23.2)
Collecting pytest-timeout>=2.4.0 (from kaleido)
  Downloading pytest_timeout-2.4.0-py3-none-any.whl.metadata (20 kB)
Collecting simplejson>=3.19.3 (from choreographer>=1.1.1->kaleido)
  Downloading simplejson-3.20.2-cp312-cp312-win_amd64.whl.metadata (3.4 kB)
Requirement already satisfied: pytest>=7.0.0 in c:\users\purva\appdata\roaming\python\python312\site-packages (f
rom pytest-timeout>=2.4.0->kaleido) (9.0.2)
Requirement already satisfied: colorama>=0.4 in c:\users\purva\appdata\roaming\python\python312\site-packages (f
rom pytest>=7.0.0->pytest-timeout>=2.4.0->kaleido) (0.4.6)
Requirement already satisfied: iniconfig>=1.0.1 in c:\users\purva\appdata\roaming\python\python312\site-packages
(from pytest>=7.0.0->pytest-timeout>=2.4.0->kaleido) (2.3.0)
Requirement already satisfied: pluggy<2,>=1.5 in c:\users\purva\appdata\roaming\python\python312\site-packages (
from pytest>=7.0.0->pytest-timeout>=2.4.0->kaleido) (1.6.0)
Requirement already satisfied: pygments>=2.7.2 in c:\users\purva\appdata\roaming\python\python312\site-packages
(from pytest>=7.0.0->pytest-timeout>=2.4.0->kaleido) (2.19.1)
Downloading kaleido-1.2.0-py3-none-any.whl (68 kB)
Downloading choreographer-1.2.1-py3-none-any.whl (49 kB)
Downloading logistro-2.0.1-py3-none-any.whl (8.6 kB)
Downloading pytest_timeout-2.4.0-py3-none-any.whl (14 kB)
Downloading simplejson-3.20.2-cp312-cp312-win_amd64.whl (75 kB)
Installing collected packages: simplejson, logistro, pytest-timeout, choreographer, kaleido

   ---------- 0/5 [simplejson]
   ---------- ---------- 3/5 [choreographer]
   ---------- ------- 4/5 [kaleido]
   ---------- 5/5 [kaleido]

Successfully installed choreographer-1.2.1 kaleido-1.2.0 logistro-2.0.1 pytest-timeout-2.4.0 simplejson-3.20.2
Note: you may need to restart the kernel to use updated packages.
[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [8]:

```python
fig_map = px.choropleth(
    country_stats,
    locations='iso_alpha',
    color='mean_temp',
    hover_name='country',
    hover_data={
        'mean_temp': ':.1f',
        'mean_humidity': ':.1f',
        'mean_precip': ':.2f',
        'record_count': True,
        'iso_alpha': False,
    },
    color_continuous_scale='RdYlBu_r',
    title='Global Mean Temperature by Country',
    labels={'mean_temp': 'Mean Temp (°C)'},
)
fig_map.update_layout(
    geo=dict(showframe=False, showcoastlines=True, projection_type='natural earth'),
    title_font_size=20, width=1200, height=600,
)

# Save as interactive HTML
html_path = os.path.join(FIGURES_DIR, 'spatial_temp_map.html')
fig_map.write_html(html_path)
print(f'Interactive map saved → {html_path}')

# Try saving as PNG too
try:
    png_path = os.path.join(FIGURES_DIR, 'spatial_temp_map.png')
    fig_map.write_image(png_path, width=1200, height=600)
    print(f'Static map saved → {png_path}')
except Exception as e:
    print(f'Could not save static PNG (kaleido needed): {e}')

fig_map.show()
print('\n✅ Unique analyses complete!')
```

```
Interactive map saved → c:\Users\purva\OneDrive\Desktop\PMA\outputs\figures\spatial_temp_map.html
Static map saved → c:\Users\purva\OneDrive\Desktop\PMA\outputs\figures\spatial_temp_map.png
```